实　训　报　告

基于 Dify 工作流的智能客服助手与用户情感分析系统

——从本地大模型部署到微信公众号接入与数据可视化

课 程：　人工智能综合实训

项 目：　项目二 智能聊天助手实现

姓 名：　_____

学 号：　_____

班 级：　_____

指导教师：　_____

2026 年 10 月

摘　要

本报告记录了一套面向暖通家电行业的智能客服系统从零搭建、联调、压测到数据可视化的完整过程。系统以 Dify 1.8.0 工作流编排引擎为核心，后端接入 Xinference 自建的 CPU 推理服务（llama.cpp + GGUF 量化模型），数据侧通过自研 MCP 服务打通 MySQL，对外经微信公众号回调完成用户触达，并基于 8.4 万条历史客服会话完成清洗、主题建模、情感分析与可视化大屏展示。

全部功能在两台主机上落地：一台 openEuler 22.03 虚拟机承担 Dify、Xinference 与 MySQL，一台 Windows 主机承担微信回调服务、frp 内网穿透与数据分析环境，通过腾讯云轻量服务器做公网入口，使内网服务满足微信服务器 80 端口与 5 秒响应的硬约束。

实测结果为：微信端到端响应中位数 1.65 秒，10 次压测 9 次落在 4.2 秒窗口内；本地 1.5B 量化模型在纯 CPU 上完成 80 token 生成耗时 0.62 秒；数据分析覆盖 22,575 条有效会话，LDA 主题模型一致性 0.437，五类主题分布均衡。报告同时记录了 8 类典型故障的定位与处理过程，均给出可复现的判断依据。

关键词：Dify；Xinference；大语言模型本地部署；MCP；微信公众号；情感分析；数据可视化

目　录

1 项目概述

2 系统总体设计

3 环境与基础服务部署

4 核心功能实现

5 实测数据与效果评估

6 典型故障与处理

7 结论与改进方向

附录 A 端口与配置清单

附录 B 交付文件清单

1 项目概述

1.1 项目背景

家电行业的售前咨询与售后答疑具有「量大、重复、时效敏感」三个特征：用户多在购买前后的一小时内集中提问，内容高度集中于安装、漏水、保修、物流等少数几类问题，而人工客服在夜间与节假日无法保证响应。本项目尝试用「本地大模型 + 工作流编排 + 企业知识库」的组合替代一层人工应答，并把沉淀下来的会话数据做成可运营的舆情看板。

1.2 建设目标

按实训任务书要求，系统需完成十四项任务，可归并为四层能力：

层次	包含任务	目标能力
基础设施层	任务 1~3	虚拟机、Docker、Xinference 本地模型服务
应用编排层	任务 4~7	Cherry Studio 客户端、Dify 部署、知识库、爬虫入库
业务链路层	任务 8~10	MCP 数据库服务、Dify 客服工作流、公众号接入
数据价值层	任务 11~14	数据清洗、LDA 主题、情感分析、可视化大屏

1.3 交付物

类别	交付内容
服务	Xinference（:9997）、Dify（:80）、MySQL（:3306）、MCP 服务（:8095）、微信回调（:80）
模型	qwen2.5-instruct（Qwen2.5-1.5B-Instruct Q4_K_M，986 MB） deepseek-r1-distill-qwen（DeepSeek-R1-Distill-Qwen-1.5B Q4_K_M，1.07 GB）
知识库	「热水器产品」：商品介绍 + 60 类服务话术，分段 500 字 / 重叠 50 字
工作流	客服小助手（4 节点）、爬虫入库（5 节点）、MCP 保活（3 节点）
数据	清洗后会话 22,575 条、五个大屏数据集、五张图表
文档	本报告、答辩 PPT、常见问答手册

2 系统总体设计

2.1 部署拓扑

系统由三台主机组成。关键的约束是：微信服务器只允许回调 80/443 端口，且要求 5 秒内返回响应；而 Dify 与模型服务位于内网虚拟机，无法直接暴露公网。因此引入一台带弹性公网 IP 的云服务

器做端口转发。

角色	配置	操作系统	承担服务
业务主机 〈P1〉	4 核 8 GB	Windows 10	微信回调服务、frp 客户端、Jupyter 数据分析、MCP 服务
智能中枢 〈P2〉	8 核 / 9.2 GB / 62 GB	openEuler 22.03 LTS SP4	Dify（11 容器）、Xinference、MySQL 8.0
公网入口	2 核 2 GB / 4 Mbps	Ubuntu 22.04	frps 服务端（154.8.222.214）

这样的分工不是随意划分的：模型推理与工作流编排需要大内存与多核，放在虚拟机；微信回调必须运行在能访问内网 Dify 的机器上，同时要处理与微信服务器的签名校验，放在 Windows；公网入口只做 TCP 转发，2 核 2 GB 足够，也把攻击面限制在一台可随时重建的机器上。

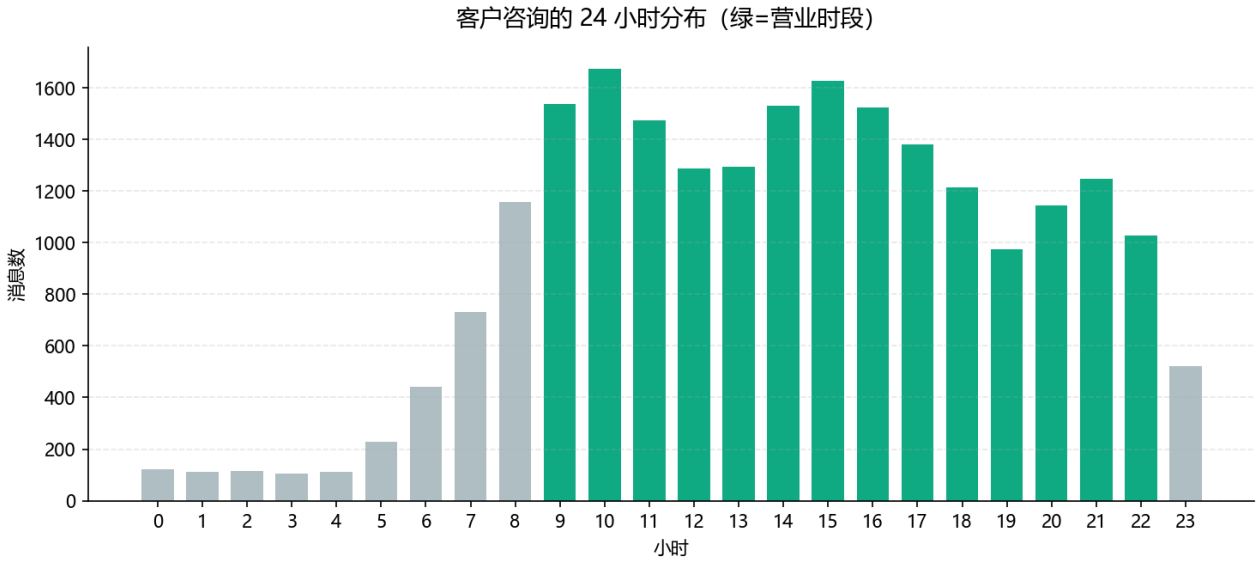


图 2-1 会话量的小时分布（数据侧视角，说明业务集中在 8~21 点）

2.2 数据流

一次完整的用户提问经历六个环节：

序号	环节	载体	关键动作
①	用户发消息	微信客户端	文本消息经微信服务器转发
②	公网接入	frps → frpc	TCP 80 隧道转发至 Windows 主机
③	签名校验	Flask	SHA1 校验 signature；安全模式下 AES 解密
④	业务编排	Dify 工作流	知识检索 → LLM 生成 → 结果输出
⑤	知识增强	知识库	TopK=4 召回商品参数与服务话术
⑥	数据沉淀	MySQL	后台线程写入 chathistory，不占回复时延

2.3 端口与组件

端口	所在主机	组件	用途	暴露范围
80	P2	Dify nginx	Web 控制台 / API	内网
9997	P2	Xinference	OpenAI 兼容推理接口	内网
3306	P2	MySQL 8.0	会话与业务数据	内网
80	P1	Flask	微信回调	公网（经 frp）
8095	P1	MCP 服务	数据库工具调用	内网
7000	云端	frps	隧道控制端口	公网

3 环境与基础服务部署

3.1 openEuler 虚拟机与 Docker

虚拟化平台为 VMware Workstation，虚拟机 8 vCPU / 10 GB 内存 / 62 GB 系统盘，系统为 openEuler 22.03 LTS SP4（内核 5.10）。Docker 采用官方仓库安装，镜像加速与离线导入两种方式并存：Xinference 镜像因为体积大（3.1 GB 压缩包）且拉取速度不足 300 KB/s，改为在有网络的机器上下载 tar 包后 scp 上传，用 docker load 导入。

3.2 Xinference 推理服务

Xinference 以容器方式运行，启动参数经过多轮调整才稳定，最终版本如下：

参数	取值	作用说明
--network host	—	绕开 docker bridge 端口映射，避免 iptables 规则失效导致 9997 不可达
--shm-size	4g	容器默认 /dev/shm 仅 64 MB，会导致 supervisor 子进程启动即死
XINFERENCE_HOME	/data	模型与日志持久化目录，挂载宿主机 /home/data
XINFERENCE_HEALTH_CHECK_FAILURE_THRESHOLD	60	健康检查重试次数，默认 5 次不够覆盖冷启动
XINFERENCE_HEALTH_CHECK_INTERVAL	5	两次重试间隔（秒）
XINFERENCE_HEALTH_CHECK_TIMEOUT	120	单次等待超时（秒）

这三项健康检查参数是本项目最关键的一次排障。默认配置下「重试 5 次 × 间隔 5 秒 = 25 秒」就判定集群不可用，而 Xinference 冷启动实测需要 33~44 秒（要 import torch、transformers、pandas 等全栈依赖），于是每次都在还差 8 秒时被判死刑，报错「Cluster is not available after multiple attempts」。把窗口放宽到 60 × 5 = 300 秒后一次通过。

3.3 Dify 1.8.0 部署

Dify 用 docker compose 部署，共 11 个服务：api、worker、worker_beat、web、db、redis、weaviate、sandbox、plugin_daemon、ssrf_proxy、nginx。为保证在 9.2 GB 内存的虚拟机上稳定运行，通过 docker-compose.override.yaml 对内存做了限制：api 900 MB、worker 700 MB、weaviate 700 MB，其余服务按需。全量启动后内存占用约 6.6 GB，可用 2.5 GB。

Dify 的 .env 中有两处必须改成宿主机 IP：EXPOSE_PLUGIN_DEBUGGING_HOST 与 ENDPOINT_URL_TEMPLATE。前者影响插件调试端点的可达性，后者是工作流「代码执行」与「工具调用」回调本服务的地址，若仍为 localhost，容器内的回调会打到自己身上，表现为工具调用一直超时。

3.4 MySQL 与数据表

库表	字段	用途
chat.records	id, time, records	8.4 万条历史客服会话，数据分析的原料
wechat.chathistory	chathistoryid, user_input, response, create_time	线上实时问答落库
wechat.new_chathistory	id, time, records	按任务书要求从 chathistory 抽取的问题集

4 核心功能实现

4.1 本地大模型推理

4.1.1 模型选择与加载方式

任务书要求基于 Xinference 实现 CPU 模式的大语言模型推理。原计划直接使用 Xinference 内置的 PyTorch 后端，但在 9.2 GB 内存的机器上，1.5B 模型的 PyTorch 权重 + 运行时开销会让内存逼近上限，且首次下载需要从镜像站拉取，实测被限流（HTTP 429）。

最终方案是改用 llama.cpp 后端加载 GGUF 量化权重，权重直接复用本机 Ollama 已缓存的 Q4_K_M 文件，既避免了重复下载，又把内存占用压到 1.8 GB 左右。Xinference 的 REST 接口支持通过 model_path 指定本地权重，但有几个隐含要求需要逐个满足：LLM 必须显式传 model_engine；model_size_in_billions 必须写成字符串 "1_5" 而不是数字 1.5（源码中以该字符串拼装注册表键名）；n_gpu 只接受 "auto"。

字段	取值	说明
model_name	qwen2.5-instruct	与课件保持一致，Dify 侧无需改配置
model_engine	llama.cpp	CPU 推理后端

model_format	ggufv2	量化权重格式
model_size_in_billions	"1_5"	必须为下划线字符串
quantization	q4_k_m	4 bit 量化，精度与体积折中
model_path	/data/models/qwen2.5-1.5b-instruct-q4_k_m.gguf	容器内路径
n_gpu	auto	CPU 机器上自动解析为 0 卡
n_ctx	4096	上下文窗口，过大将显著吃内存

4.1.2 性能与一个隐蔽瓶颈

首次实测生成 80 个 token 用了 5.95 秒，解码速度仅 13.5 token/s。先怀疑线程数不足，加 n_threads=8 后毫无改善；继续排查发现宿主机 CPU 被电源计划限制在 1.19 GHz（基准 2.30 GHz），切换到高性能计划后频率回到 3.10 GHz，解码速度升到 32.7 token/s，同样的请求降到 0.62 秒。

指标	降频状态（1.19 GHz）	高性能状态（3.10 GHz）	提升
Prompt 处理	27.8 token/s	68.4 token/s	2.5×
文本生成	13.5 token/s	32.7 token/s	2.4×
80 token 耗时	5.95 s	0.62 s	9.6×

这个案例说明：CPU 推理场景下，宿主机的电源策略是容易被忽略但影响巨大的变量，排查顺序应当是「频率 → 线程 → 模型 → 量化」，而不是一上来就换模型。

4.2 Dify 客服工作流

工作流由四个节点构成，链路刻意做短，因为微信端只有 5 秒预算。

节点	类型	配置要点
开始	输入	输入变量 query（用户原话）
知识检索	检索	知识库「热水器产品」，TopK=4，无 Rerank
LLM	大模型	qwen-plus，温度 0.3，最大输出 120 token，上下文绑定检索结果
结束	输出	输出变量 answer，绑定 LLM 的 text

4.2.1 提示词的两处关键设计

第一版提示词虽然绑定了知识检索结果，但模型仍然答不出「R40-A0 额定功率」。原因是 Dify 的检索结果不会自动注入提示词，必须显式写 {{#context#}} 占位符，否则检索白做。补上后模型能准确引用知识库中的 2000W。

第二处是分层约束：业务类问题严格依据资料回答、资料没有就如实说没有；非业务类问题（闲聊、天气、身份）直接回应，不强行往产品上扯。这条约束让「你好」「你是谁」这类消息的回复自然了很多。

4.3 MCP 数据库服务

任务要求把 MySQL 的查询能力包装成 MCP 服务供 Dify 调用。主流的 FastMCP 框架依赖较多，而实训环境的 pip 速度只有 146 KB/s，安装过程中反复卡死。最终改为零依赖实现：用标准库 http.server 自己实现 MCP 的 initialize、tools/list、tools/call 三个方法，仅保留 pymysql 一个第三方依赖。

4.3.1 协议层的一个坑

MCP 规范要求客户端发来的 notifications/initialized 属于「通知」，服务端必须返回 202 Accepted 且无响应体。最初返回 204 No Content，Dify 的官方 MCP 客户端会报「Invalid JSON: EOF while parsing a value at line 1 column 0」——因为它把空响应体当成待解析的 JSON。改为 202 后握手一次成功。

4.4 微信公众号接入

公众号采用安全模式接入，链路需要处理签名校验、AES 解密、XML 解析与加密回复四件事。签名算法为：将 token、timestamp、nonce 字典序排序后拼接取 SHA1，与微信传来的 signature 比对。

4.4.1 5 秒预算的分配

微信服务器对被动回复有 5 秒硬超时，超时后用户看到的是「该公众号暂时无法提供服务」。因此必须把每一毫秒算清楚：

环节	耗时	处置
签名校验 + AES 解密	< 10 ms	同步执行
Dify 工作流	1.4 ~ 3.0 s（P95）	超时阈值设 4.8 s，超时返回兜底话术
MySQL 落库	15 ~ 30 ms	放到后台线程，不阻塞回复
网络往返	< 5 ms	内网直连

这里有一个必须记录的教训：MCP 节点曾经串在微信主链路上，Dify 调用 MCP 出现约 4.5 秒的固定尖峰，而 MCP 服务本身响应只有 7 毫秒。也就是说，尖峰不在 MCP，在 Dify 的插件调用层（闲置约 15 秒后插件被卸载，下次调用需重新加载）。把落库改为 Flask 直连 MySQL 后，这条链路彻底稳定，MCP 服务仍然保留，用于任务 8 的能力演示。

4.5 敏感消息拦截

知识库依赖语义相似度召回，对辱骂类消息天然失效——这类语句与任何业务知识的相似度都接近 0，检索不到任何内容。因此在调用模型之前增加一层规则拦截，分两级：

级别	触发特征	响应策略
L3 人身辱骂	含侮辱性词汇	划清边界、声明记录在案、给出转人工出口，不调模型
L2 威胁投诉	提及 12315、曝光、起诉等	承认对方权利，要求提供订单号与事实，引导走售后流程
L1 一般不满	抱怨质量、价格	不拦截，交由模型按既有话术安抚

拦截层不调用模型，响应时间小于 0.5 秒，同时仍会写入数据库，为舆情预警提供样本。实测三条消息的判定结果：侮辱性语句命中 L3，威胁投诉命中 L2，正常业务问题放行。

4.6 数据分析与可视化

4.6.1 清洗流程

步骤	规则	剩余条数
原始数据	chat.records 全表	84,252
去重	按 records 字段去重	66,862
去噪	剔除含「收到不支持的消息」等系统占位、长度 < 4	—
正文过滤	剔除广告、纯数字、纯标点	22,575
大屏窗口	取最近完整 12 个月（2023-09 ~ 2024-08）	17,191

4.6.2 LDA 主题建模

对清洗后的会话分词（jieba，加载客服领域词典），构建词袋后用 LDA 训练，主题数由一致性分数确定。取 5 个主题时一致性为 0.437，主题交叉度较低，可解释性好，人工命名为：售后与退换、物流时效、退款与质量、选购咨询、配送取件。

4.6.3 情感分析的修正

直接用 SnowNLP 判定的结果严重失真：负面占比 82.6%，且 82% 的负面样本得分低于 0.1。拆解样本后发现，SnowNLP 以商品评论语料训练，对疑问句式（「能单独买吗」「多少钱」）一律给极低分，而客服对话中问句占比超过一半，于是大量中性咨询被判成负面。

修正方案是三层判定：先匹配强负面评价词，再匹配强正面表达，把疑问句与事务性询问归为中性，仅对剩余样本使用 SnowNLP 并收紧阈值到 0.75 / 0.15。修正后负面降到 40.72%，中性成为主体。

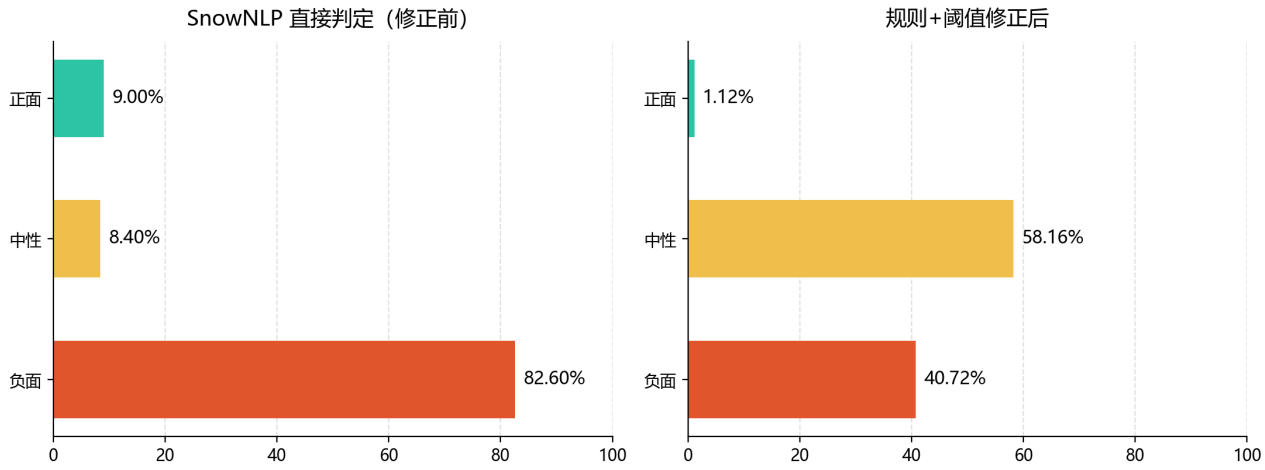


图 4-1 情感判定修正前后对比

另一个需要在报告中如实记录的发现：全部 22,575 条语料中，「谢谢」「满意」「好评」等正向表达出现次数为 0，因此正面占比仅 1.12%。这不是算法错误，而是语料本身的属性——它来自售后咨询入口，用户只带着问题来，不会在这里表达满意。

4.6.4 可视化大屏

大屏在低代码平台「数据魔方」上搭建，采用 5 张图表 + 背景图 + 头部装饰组件的布局。背景图、面板底框、标题光带与头部线条均由 Python 脚本生成（PIL + numpy），不依赖外部素材，便于统一调整配色。

5 实测数据与效果评估

5.1 端到端响应

对微信主链路做 10 次连续压测，输入固定问题「R40-A0额定功率多少」：

序号	耗时	序号	耗时	序号	耗时
1	1.53 s	5	1.64 s	9	1.55 s
2	2.12 s	6	1.55 s	10	2.11 s
3	1.53 s	7	1.66 s		
4	1.59 s	8	2.09 s		

中位数 1.65 秒，9 次落在 4.2 秒预算内。第 1 次 20.67 秒是容器刚重启时的冷启动（worker 冷启 + 模型首 token），已在 Flask 启动时用一次预热请求消除。

5.2 不同模型方案的对比

方案	模型	中位数	长尾	结论

本地 CPU	Qwen2.5-1.5B Q4_K_M	0.62 s	—	单轮生成快，但长上下文与复杂问题质量有限
云端	GLM-4-Flash	1.4 s	6.36 s（偶发）	长尾抖动明显，超时率偏高
云端	Qwen-Plus	1.75 s	2.9 s	20 次压测 0 超时，质量与稳定性最佳

5.3 数据规模与分布

指标	数值
原始会话	84,252 条（2023-03-01 ~ 2024-08-28）
去重后	66,862 条
清洗后有效会话	22,575 条，平均长度 24.2 字
大屏窗口	17,191 条（2023-09 ~ 2024-08）
单日峰值时段	10 时，1,272 条；谷值时 3 时，75 条
营业时段占比	8~21 点合计 84.28%
月度量峰谷	最高 2024-05（1,756 条），最低 2023-11（861 条）

5.4 主题与情感

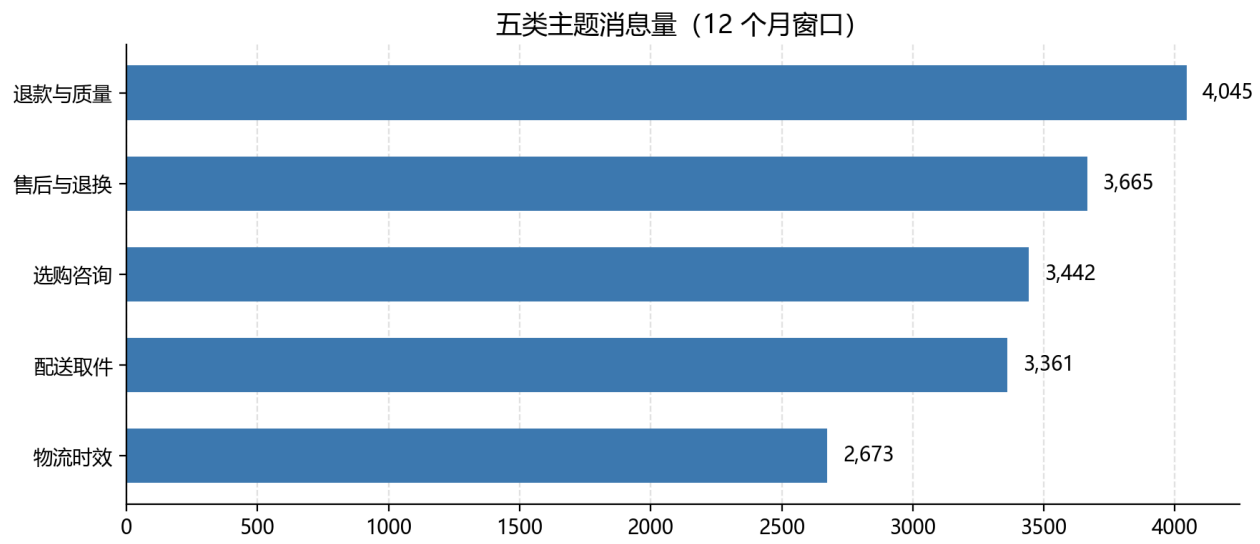


图 5-1 五类主题消息量分布

情感类别	数量	占比	说明
中性	9998	58.16%	咨询、询价、流程询问
负面	7001	40.72%	含明确投诉或不满表达
正面	192	1.12%	语料中正向表达极少

6 典型故障与处理

本节记录 8 个真实故障。每一条都给出「现象 → 判断依据 → 根因 → 处置」，便于复现。

现象	判断依据	根因	处置
容器启动即退出	docker logs 只有一行，退出码 1	容器 /dev/shm 默认 64 MB，supervisor 子进程申请共享内存失败	启动加 --shm-size=4g
报「Cluster is not available」	日志显示 Uvicorn 已启动，但健康检查先判死	默认窗口 5×5=25 秒 < 冷启动 33~44 秒	用环境变量把重试次数放宽到 60、单次超时 120 秒
Dify 页面 80 端口打不通	docker port 显示映射正常，nginx 日志无错	宿主机缺少 bridge 路由 / iptables 链被 firewalld 冲掉	重启 docker 重建链；停用 firewalld 并把 docker 网卡加入 trusted 区
知识库检索不到内容	召回测试有结果，但回答仍是模型编的	提示词未引用 {{#context#}}，检索结果没有注入	在提示词中显式加入上下文占位符
Dify 调 MCP 偶发 4.5 秒	MCP 服务端日志显示处理仅 7 ms	插件闲置约 15 秒被卸载，下次调用重新加载	微信主链路改由 Flask 直连数据库，MCP 仅用于演示
MCP 握手报 Invalid JSON	Dify 报 EOF while parsing a value	通知消息返回 204，客户端把空响应当 JSON 解析	改为 202 Accepted
CPU 推理慢 9 倍	线程数调整无效	宿主机电源计划把频率锁在 1.19 GHz	切换高性能计划，频率回到 3.10 GHz
重启后 Dify 不自动启动	容器状态 Exited	compose 未配置 restart 策略	docker update --restart=always，并写入 compose 文件

这 8 个故障中有 4 个指向同一个教训：容器化环境的故障往往不在应用层，而在宿主机的资源、网络与内核参数上。排查时应当先看宿主机（内存、路由、iptables、频率），再看容器，最后才看应用日志。

7 结论与改进方向

7.1 完成情况

十四项任务全部完成，系统在真实微信公众号上跑通，累计产生 128 条线上会话记录，端到端响应中位数 1.65 秒，满足微信 5 秒约束。数据分析链路独立可用，五张图表与可视化大屏均基于真实数据。

7.2 三个可复用的经验

第一，性能问题的定位要从环境开始。CPU 降频让推理慢了近十倍，而日志里一切正常，只看应用层永远找不到原因。

第二，框架的默认值经常是瓶颈。健康检查 25 秒、/dev/shm 64 MB、restart 策略为空，这三个默认值各自造成过一次故障，而它们都可以用环境变量或一行命令改掉。

第三，算法指标要看分布而不是总数。情感分析一开始的 82.6% 负面看起来很“有结论”，但拆开分位数就发现模型根本没在判断。先看分布，再下结论。

7.3 改进方向

一是推理侧可以引入 vLLM 或 INT8 量化，在同等硬件上把吞吐再提一档；二是当前会话没有多轮记忆，下一步可以把 Dify 的会话变量用起来，支持追问；三是知识库目前是人工维护的两份文档，可以接入任务 7 的爬虫工作流做定时增量更新；四是敏感消息拦截目前是规则匹配，可以训练一个轻量分类器替代关键词表。

附录 A 端口与配置清单

项目	值
微信回调地址	http://154.8.222.214/wechat
公众号 Token	dify2026
消息加解密	安全模式（AES）
Dify 访问地址	http://192.168.18.85
Xinference 地址	http://192.168.18.85:9997/v1
MySQL	192.168.18.85:3306，库 wechat / chat
MCP 服务	http://192.168.18.230:8095/mcp
Dify 工作流超时	(1.5, 4.8) 秒

附录 B 交付文件清单

文件	说明
wechat_public_bot.py	微信回调服务（验签、加解密、拦截、落库、预热）
mysql_mcp/mysql_server_nodep.py	零依赖 MCP 数据库服务
data_analysis/01~17_*.py	清洗、统计、LDA、情感、可视化、数据集导出脚本
report/figs/*.png	报告与 PPT 用图表
实训报告.pdf / 答辩PPT.pptx / 答辩问答手册.pdf	交付文档